```
"Hour is #{hour}, minute #{min}"               # => "Hour is 12, minute 50"

# You can mix named and position-based references
"Hour is #{hour}, minute #{$2}"                # => "Hour is 12, minute 50"
```

Finally, the named matches also can be used as indexes into a MatchData instance.

tut_regexp/named_regex_groups_3.rb

```
md = /(?<hour>\d\d):(?<min>\d\d)(..)/.match("12:50am")
"Hour is #{md[:hour]}, minute #{md[:min]}" # => "Hour is 12, minute 50"
```

As you can see in these examples, named groups are a mixed bag—they can make the actual regular expression look more complicated, but can also make expressions using the result of the match clearer.

Pattern-Based Substitution

We've already seen how sub and gsub replace the matched part of a string with other text. In those previous examples, the pattern was always fixed text, but the substitution methods work equally well if the pattern contains repetition, alternation, and grouping.

```
a = "quick brown fox"
a.sub(/[aeiou]/,  "*") # => "q*ick brown fox"
a.gsub(/[aeiou]/, "*") # => "q**ck br*wn f*x"
a.sub(/\s\S+/,  "")    # => "quick fox"
a.gsub(/\s\S+/, "")    # => "quick"
```

The substitution methods can take a string or a block. If a block is used, the block is passed each matching substring, and the block's return value is substituted into the original string.

```
a = "quick brown fox"
a.sub(/^./) { |match| match.upcase }       # => "Quick brown fox"
a.gsub(/[aeiou]/) { |vowel| vowel.upcase } # => "qUIck brOwn fOx"
```

Maybe we want to normalize city names entered by users into a web application, even if the city name is multiple words. They may enter NEW YORK, new york, or nEw yORk, and we'd like to store it as New York. The following method is a simple first iteration. The pattern that matches the first character of a word is \b\w—look for a word boundary followed by a word character. Combine this with gsub, and we can hack the names:

```
def mixed_case(name)
  name.downcase.gsub(/\b\w/) { |first| first.upcase }
end
mixed_case("NEW YORK") # => "New York"
mixed_case("new york") # => "New York"
mixed_case("nEw yORk") # => "New York"
```

As we saw previously, that substitution block could also be written as either { _1.upcase } or as a second argument written as &:upcase.

```
def mixed_case(name)
  name.downcase.gsub(/\b\w/, &:upcase)
end

mixed_case("nEw yORk") # => "New York"
```